CANLI DERS

React Temelleri

Bileşen, JSX, Props, State, olaylar ve yan etkiler: arayüz geliştirmenin temel yapı taşları.

DERS PLANI

Bugün neler var?

1. Giriş ve React'in mantığı
2. Bileşenler ve JSX
3. Props
4. State ve useState
5. Olaylar, koşullu render, listeler
6. useEffect'e giriş
7. İleri konular: state yukarı taşıma, Context, useRef, veri çekme ve form gönderme
8. Sık hatalar, özet, soru-cevap

01 · GIRIŞ

Neden React?

Bileşen tabanlı

Arayüzü tekrar kullanılabilir parçalara ayırırsın. Her parça kendi görünümünü ve mantığını taşır.

Deklaratif

Adımları tek tek yazmak yerine ekranın nasıl görünmesi gerektiğini tarif edersin. Gerisini React yapar.

Veriye bağlı

Veri değişince arayüz otomatik güncellenir. DOM'u elle değiştirmen gerekmez.

TEMEL FIKIR

UI = f(state)

Veri (state) değişince ekran ona göre yeniden çizilir. Bu cümleyi bütün ders boyunca hatırlayacağız.

Bileşen: JSX döndüren bir fonksiyon

- Adı büyük harfle başlar
- JSX (HTML benzeri işaretleme) döndürür
- Tekrar kullanılabilir: istediğin kadar çağırırsın
- Kendi görünümünü ve mantığını taşır

```
function Button() {
  return <button>Tıkla</button>;
}
```

JSX: JavaScript içinde HTML'e benzer yazım

```
const ad = "Ayşe";
const selam = <h1 className="baslik">Merhaba, {ad}!</h1>;
```

- **class** yerine **className** yazılır
- **{ }** içine herhangi bir JavaScript ifadesi konur: {2 + 3}, {ad}
- Birden fazla kök eleman için **<> ... </>** (Fragment) kullanılır

Küçük parçalar, büyük ekran

```
function Baslik() { return <h1>Ders Notları</h1>; }
function Kart() { return <p>Bu bir kart.</p>; }

function App() {
  return (
    <main>
      <Baslik />
      <Kart />
      <Kart />
    </main>
  );
}
```

Ağaç yapısı: **App** → Baslik, Kart, Kart. Bileşen içinde bileşen çağırarak sayfayı katman katman kurarsın.

Props: bileşene dışarıdan veri vermek

```
function Selamla({ ad }) {
  return <p>Merhaba, {ad}!</p>;
}

function App() {
  return (
    <Selamla ad="Ayşe" />
    <Selamla ad="Mehmet" />
  );
}
```

- Props, HTML özelliği gibi yazılır: **ad=“Ayşe”**
- Props salt okunurdur: bileşen kendi props’unu değiştirmez
- Veri üst bileşenden alt bileşene doğru akar

Özel bir prop: children

```
function Kutu({ children }) {
  return <div>{children}</div>;
}

<Kutu>
  <p>Bu içerik kutunun içine girer.</p>
</Kutu>
```

Varsayılan değer

Prop gelmezse bir değer ata: **{ renk = “mavi” }**

Tek yönlü akış

Veri yukarıdan aşağıya akar. Alt bileşen üst bileşenin verisini doğrudan değiştiremez.

06 · STATE

Props dışarıdan gelir, state bileşenin hafızasıdır

	Props	State
Kim verir?	Üst bileşen	Bileşenin kendisi
Değişir mi?	Hayır, salt okunur	Evet, setter ile
Değişince?	Yeniden çizilir	Yeniden çizilir

Ekranda zamanla değişen bir şey (sayaç, açık/kapalı menü, yazılan metin) state olmalıdır.

useState ile bileşen hafızası

```
import { useState } from "react";

function Sayac() {
  const [sayi, setSayi] = useState(0);

  return (
    <button onClick={() => setSayi(sayi + 1)}>
      Tıklama: {sayi}
    </button>
  );
}
```

- **[sayi, setSayi]**: değer ve onu güncelleyen fonksiyon. Başlangıç değeri 0
- **setSayi** çağrılınca bileşen yeniden çizilir
- State'i doğrudan değiştirme (**sayi = 5** yazma), her zaman setter kullan

Fonksiyonu çağırma, ver

Yanlış: hemen çalışır

```
<button onClick={sayacArtir()}>
```

Doğru: tıklanınca çalışır

```
<button onClick={sayacArtir}>
```

- Olay adları: **onClick**, **onChange**, **onSubmit**
- Parametre gerekiyorsa sarmala: **onClick={() ⇒ sayacArtir(5)}**
- React olay nesnesini (event) handler'a otomatik verir

Kontrollü input: değer state'te yaşar

```
function AdAlani() {
  const [ad, setAd] = useState("");

  return (
    <div>
      <input value={ad} onChange={(e) ⇒ setAd(e.target.value)} />
      <p>Merhaba, {ad}</p>
    </div>
  );
}
```

- **value**: ekranda görünen değer her zaman state'ten gelir
- **onChange**: her tuş vuruşunda state güncellenir
- İkisi birlikte: input ile state her zaman aynı değeri gösterir

Koşula göre ne gösterileceğine karar ver

```
function Mesaj({ giris }) {
  return (
    <div>
      {giris ? <p>Hoş geldin!</p> : <p>Lütfen giriş yap.</p>}
      {giris && <button>Çıkış</button>}
    </div>
  );
}
```

Üçlü ifade

İki durumdan birini seçer: **? :**

&& kısa yolu

Koşul doğruysa göster, değilse hiçbir şey gösterme.

map ile çiz, key ile tanıt

```jsx
function Liste() {
  const meyveler = ["Elma", "Armut", "Muz"];

  return (
    <ul>
      {meyveler.map((m) => (
        <li key={m}>{m}</li>
      ))}
    </ul>
  );
}
```

- **key** her öğe için benzersiz olmalı; React hangi öğenin değiştiğini bunla anlar
- Sıralaması değişebilen listelerde index'i key olarak kullanmak hataya yol açar

useEffect: render dışındaki işler

```
useEffect(() => {
  document.title = `Sayı: ${sayi}`;
}, [sayi]);
```

- Veri çekme (fetch), zamanlayıcı, başlık güncelleme gibi işler için kullanılır
- **[ ]** boşsa sadece ilk çizimde çalışır
- **[sayi]** yazarsan sayi değiştikçe tekrar çalışır

İki kardeş bileşen aynı veriye ihtiyaç duyunca

```
function App() {
  const [sayi, setSayi] = useState(0);
  return (
    <>
      <Goster sayi={sayi} />
      <Artir onArtir={() => setSayi(sayi + 1)} />
    </>
  );
}
```

- Ortak veri, en yakın ortak üst bileşende tutulur
- Alt bileşenler veriyi props ile alır
- Değişikliği **onArtir** gibi bir fonksiyon props'uyla üste bildirirler

Props'u her kademeden geçirmek zorunda mıyız?

```
const TemaContext = createContext('acik');

function App() {
  return (
    <TemaContext.Provider value="koyu">
      <Sayfa />
    </TemaContext.Provider>
  );
}

function Dugme() {
  const tema = useContext(TemaContext);
  return <button>{tema}</button>;
}
```

- Ara bileşenler kullanmadığı props'u taşımak zorunda kalmaz (prop drilling)
- Tema, oturum bilgisi gibi genel veriler için uygundur
- Her şey için Context kullanma; önce props yeterli mi diye bak

useRef: DOM'a doğrudan erişim

```
const inputRef = useRef(null);

<input ref={inputRef} />
inputRef.current.focus();
```

- **ref.current** değişince bileşen yeniden çizilmez
- Odaklanma, kaydırma, ölçüm gibi DOM işleri için kullanılır
- Ekranda görünmesi gereken değerler için state kullan, ref değil

useEffect ile sunucudan veri çekmek

```
const API = 'https://jsonplaceholder.typicode.com/users';
function Kullanicilar() {
  const [veri, setVeri] = useState([]);
  useEffect(() => {
    fetch(API).then((r) => r.json()).then(setVeri);
  }, []);
  return <ul>{veri.map((u) => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

- **[]** boş olduğu için istek sadece ilk çizimde atılır
- Yüklenirken ve hata olduğunda ayrı bir durum göster (ödev olarak bırakılabilir)

onSubmit ve preventDefault

```
function Giris() {
  const [ad, setAd] = useState('');
  function gonder(e) {
    e.preventDefault(); // sayfanın yenilenmesini engeller
    alert(`Merhaba, ${ad}!`);
  }
  return (
    <form onSubmit={gonder}>
      <input value={ad} onChange={(e) ⇒ setAd(e.target.value)} />
      <button type="submit">Gönder</button>
    </form>
  );
}
```

- **onSubmit** formun gönderilmesini yakalar
- **preventDefault()** çağrılmazsa tarayıcı sayfayı yeniler ve state sıfırlanır

En sık yapılan 4 hata

Hata	Doğrusu
State'i doğrudan değiştirmek: sayi = 5	setSayi(5)
Listede key vermemek	Her öğeye benzersiz key
Hook'ları if veya döngü içinde çağırmak	Hook'ları bileşenin en üstünde çağır
Olayı parantezle çağırmak: onClick={f()}	onClick={f}

VS Code’da adım adım

1. Bir bileşen oluştur ve ekrana bas
2. JSX içine süslü parantezle değişken ekle
3. Props ile aynı bileşeni iki kez çağır
4. useState ile sayaç yap
5. onClick ile butonu bağla
6. Bir input ekle ve kontrollü yap
7. Listeyi map ve key ile çiz
8. useEffect ile başlığı güncelle

ÖZET

Bugün gördüğümüz temel kavramlar

Bileşen
JSX döndüren fonksiyon

JSX
JavaScript içinde HTML'e benzer yazım

Props
Dışarıdan gelen, salt okunur veri

State
useState ile bileşenin hafızası

Olaylar
onClick, onChange ile etkileşim

useEffect
Render dışı işler, dependency array ile

SORU-CEVAP

Sorular?

Teşekkürler! Şimdi kodu birlikte VS Code'da deneyelim.

Kendi bileşenini yaz

Bir kart bileşeni oluştur, props ile farklı içerik ver.

Sayaç ekle

İkinci bir useState ile bağımsız bir sayaç yap.

Formu genişlet

Kontrollü input'a ikinci bir alan ekle.

Referans: react.dev